

AetherGrid-AI Core Engine

Dynamic Cost Pricing & Traversal Engine — Code Documentation

Production Architecture: Time-Dependent Edge Pricing, Heuristic Admissibility, Anti-Flapping Filters, and Invariant Verification

Course: UCS503P (SE)

Date: September 2026

Status: Phase 01 Validated

Package: Aether/core/

Authors: Vishal Singla (1024240009) & Sparsh Verma (1024240011) **Affiliation:** Thapar Institute of Engineering & Technology

Abstract & Purpose: In emergency autonomous dispatch, the routing engine's path quality and convergence rate depend entirely on the dynamic edge pricing function $c(e, t)$. If costs fail to capture real-time bottlenecks, fleet vehicles enter gridlock; if costs fluctuate with high-frequency sensor updates, routing decisions suffer from catastrophic *route flapping*; if heuristic lower bounds overestimate travel times, A^* search forfeits mathematical optimality. This specification documents the implementation of `Aether.core.pricing` and its dependencies, providing the mathematical derivations, production code snippets, architectural dataflow schematics, formal admissibility proofs, and unit test verification.

1 System Architecture & Module Decomposition

The Aether dispatch backend is structured as a modular, time-varying directed graph system located in `Aether/core/`. The cost pricing engine (`pricing.py`) acts as the centralized evaluator connecting environmental state, vehicular density, and road topography.

1.1 Component Dependency Architecture

Figure 1 illustrates the architectural dependency hierarchy and data contracts between modules. During graph search, the traversal engine queries `calculate_edge_cost()` across open frontiers while using `compute_admissible_heuristic()` for priority queue ordering.

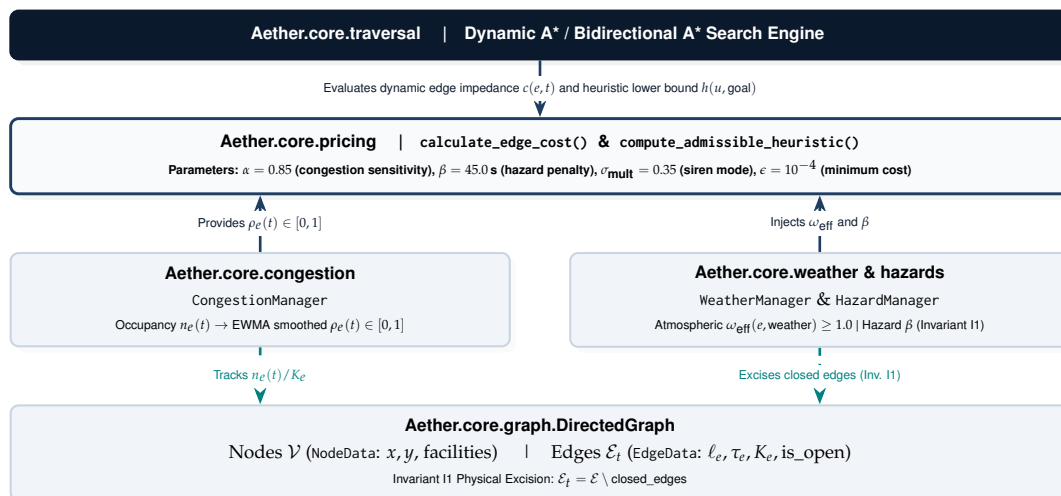


Figure 1: System Component Architecture: Layered dataflow hierarchy from traversal engine to physical graph.

1.2 Core Data Models & Graph Contracts (graph.py)

The transportation network is represented as a time-dependent directed multigraph $\mathcal{G}_t = (\mathcal{V}, \mathcal{E}_t)$ implemented via typed dataclasses in `Aether/core/graph.py`:

- **NodeData**: Represents an intersection, ambulance depot, or hospital facility $(x_u, y_u) \in \mathbb{R}^2$ with metric distance method `distance_to()`.
- **EdgeData**: Encapsulates directed corridor $e = (u, v)$ with metric length d_e , legal free-flow time ℓ_e , capacity K_e , vehicle count $n_e(t)$, smoothed congestion $\rho_e(t)$, closure flag `is_open`, and localized hazard attributes $(\beta, \mathbf{1}_{\text{hazard}})$.
- **TerrainType**: Python Enum mapping road classifications to dimensionless multipliers $\tau_e \geq 1.00$ (Table 1).

Terrain Classification	Multiplier τ_e	Free-Flow Speed	Operational Network Role & Physical Impedance
<code>TerrainType.HIGHWAY</code>	1.00	25.0 m/s (90 km/h)	Grade-separated expressways; zero pedestrian crossings; base baseline friction.
<code>TerrainType.ARTERIAL</code>	1.25	16.7 m/s (60 km/h)	Multi-lane primary transit arteries; coordinated traffic signal corridors.
<code>TerrainType.URBAN</code>	1.45	11.1 m/s (40 km/h)	Standard metropolitan city blocks; pedestrian crosswalks and turning delays.
<code>TerrainType.ALLEY</code>	1.90	5.6 m/s (20 km/h)	Narrow service alleys; strict single-lane vehicular passing restrictions.
<code>TerrainType.OFFROAD</code>	3.10	4.2 m/s (15 km/h)	Unpaved surfaces, disaster debris fields, or unpaved park access trails.

Table 1: Dimensionless terrain classification tensor τ_e implemented in `TerrainType`.

2 Dynamic Edge Cost Pricing Engine (pricing.py)

2.1 Mathematical Formulation

The instantaneous edge cost $c(e, t)$ represents expected travel impedance (in seconds):

Definition 1: Master Dynamic Edge Pricing Formulation

$$c(e, t) = \begin{cases} \max(\epsilon, \ell_e \cdot \tau_e \cdot (1 + \alpha_{\text{eff}} \cdot \rho_e(t)) \cdot w_{\text{eff}}(e, \text{weather}) + \beta \cdot \mathbf{1}_{\text{hazard}}), & \text{if } e \in \mathcal{E}_t \text{ (open)} \\ +\infty, & \text{if } e \notin \mathcal{E}_t \text{ (closed)} \end{cases} \quad (1)$$

Where parameter operational domains are calibrated as follows:

- $\ell_e = d_e / v_e$: Legal free-flow traversal time (seconds) bounded below by $\epsilon = 10^{-4}$ s to prevent zero-cost cycles.
- $\tau_e \in [1.00, 3.10]$: Road classification impedance multiplier from Table 1.
- $\alpha_{\text{eff}} = \alpha \cdot (1 - 0.65 \cdot \sigma)$: Siren preemption coefficient ($\alpha = 0.85$). Under siren mode ($\sigma = 1$), $\alpha_{\text{eff}} = 0.85 \times 0.35 = 0.2975$, reflecting vehicular yielding to audible sirens.
- $\rho_e(t) \in [0.0, 1.0]$: EWMA-smoothed traffic volume-to-capacity density ratio.
- $w_{\text{eff}}(e, \text{weather}) \geq 1.00$: Coupled atmospheric-corridor impedance multiplier (Section 3).
- $\beta \cdot \mathbf{1}_{\text{hazard}}$: Additive localized hazard delay (default $\beta = 45.0$ s).

2.2 API Specification: `calculate_edge_cost()`

Function Specification: `calculate_edge_cost()`

Signature: `calculate_edge_cost(edge, weather_multiplier=1.0, params=None, is_siren=False) -> float`

Parameters:

- `edge` (`EdgeData`): Target directed edge corridor $e = (u, v)$.
- `weather_multiplier` (`float`): Global baseline atmospheric multiplier $w_{\text{base}} \in [1.0, 2.5]$.
- `params` (`CostModelParameters`): Calibration container $(\alpha, \beta, \sigma_{\text{mult}}, \epsilon)$. Defaults to standard calibrated constants if `None`.
- `is_siren` (`bool`): Flag indicating active siren preemption.

- **Returns:** Instantaneous impedance in seconds ($\geq \epsilon$), or `float('inf')` if edge is closed.
- **Complexity:** Time $\mathcal{O}(1)$, Auxiliary Space $\mathcal{O}(1)$.
- **Invariants:** Strictly enforces Invariant I1 (returns $+\infty$ whenever `edge.is_open == False`).

2.3 Python Code Implementation: calculate_edge_cost()

Listing 1 displays the complete production implementation from Aether/core/pricing.py.

Aether/core/pricing.py (Lines 12–22, 94–138)

```

1 @dataclass(frozen=True)
2 class CostModelParameters:
3     alpha: float = 0.85          # Congestion sensitivity factor
4     default_beta: float = 45.0   # Hazard time penalty (seconds)
5     siren_multiplier: float = 0.35 # Congestion attenuation under siren mode
6     min_cost_epsilon: float = 1e-4 # Minimum positive cost bound to prevent zero-weight cycles
7
8 def calculate_edge_cost(
9     edge: 'EdgeData',
10     weather_multiplier: float = 1.0,
11     params: Optional[CostModelParameters] = None,
12     is_siren: bool = False,
13 ) -> float:
14     """Computes instantaneous temporal cost c(e, t) in seconds. Returns inf if closed."""
15     if not edge.is_open:
16         return float("inf")
17
18     p = params or CostModelParameters()
19     ell_e = max(p.min_cost_epsilon, edge.free_flow_time_seconds)
20     tau_e = edge.terrain.multiplier
21     rho_e = max(0.0, min(1.0, edge.congestion))
22     alpha_eff = p.alpha * (p.siren_multiplier if is_siren else 1.0)
23     omega_eff = _weather_corridor_factor(edge, weather_multiplier)
24
25     dynamic_travel_time = ell_e * tau_e * (1.0 + alpha_eff * rho_e) * omega_eff
26
27     hazard_cost = 0.0
28     if edge.is_hazard:
29         hazard_cost = max(0.0, edge.hazard_penalty_seconds or p.default_beta)
30
31     total_cost = dynamic_travel_time + hazard_cost
32     return max(p.min_cost_epsilon, total_cost)

```

Listing 1: Production implementation of calculate_edge_cost() in Aether/core/pricing.py.

2.4 Computational Flowchart

Figure 2 details the exact decision tree and algebraic pipeline executed inside calculate_edge_cost().

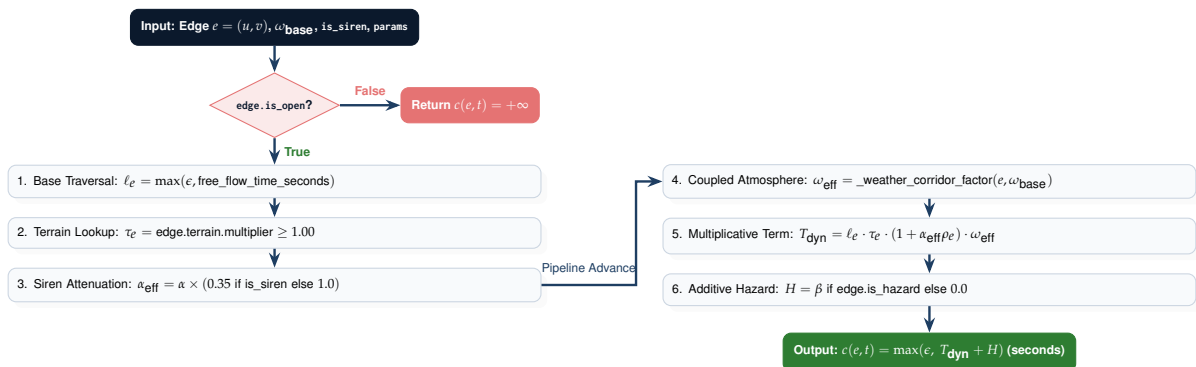


Figure 2: Algorithmic pipeline of calculate_edge_cost() partitioned into invariant verification, friction, and hazard aggregation.

3 Coupled Atmospheric-Corridor Impedance (_weather_corridor_factor)

In GIS navigation, treating weather as an uncoupled uniform scalar $\omega \geq 1.0$ across the entire graph merely inflates edge weights uniformly without altering the optimal topological route. In physical cities, emergency municipal operations deploy snowplows and salting trucks exclusively to designated primary arterial spines. Secondary residential streets accumulate heavy slush drifts, and low-lying riverfront expressways experience drainage backup.

3.1 Corridor Coupling Implementation

Listing 2 shows the corridor-specific dispatch coupling logic implemented in `Aether/core/pricing.py`.

Aether/core/pricing.py (Lines 25–35, 38–91)

```
1  _SNOW_PRIORITY_CORRIDORS = ("central park west", "broadway", "5th avenue", "8th avenue",
2                               "park avenue", "madison avenue", "west 57th street", "canal street")
3  _RAIN_FLOOD_CORRIDORS = ("columbus avenue", "11th avenue", "12th avenue", "fdr drive", "west street")
4
5  def _weather_corridor_factor(edge: 'EdgeData', weather_multiplier: float) -> float:
6      """Computes corridor-specific atmospheric impedance omega_eff(edge, weather)."""
7      omega_base = max(1.0, weather_multiplier)
8      if omega_base <= 1.01:
9          return 1.0 # Clear skies: zero penalty
10     st_name = (getattr(edge, "street_name", None) or edge.closure_reason or "").lower()
11
12     # 1. Snow / Blizzard Storm (omega_base >= 1.60)
13     if omega_base >= 1.60:
14         if any(p in st_name for p in _SNOW_PRIORITY_CORRIDORS):
15             return 1.10 # Salted and plowed primary emergency spine
16         elif edge.terrain == TerrainType.HIGHWAY:
17             return 2.50 # Elevated highway bridge decks freeze over severely
18         elif st_name:
19             return 2.15 # Unplowed secondary side avenues accumulate slush
20         return omega_base
21
22     # 2. Rain / Severe Storm (1.15 < omega_base < 1.35)
23     elif 1.15 < omega_base < 1.35:
24         if any(p in st_name for p in _RAIN_FLOOD_CORRIDORS):
25             return 1.85 # Low-lying waterfront avenues experience flash ponding
26         elif edge.terrain in (TerrainType.ALLEY, TerrainType.OFFROAD):
27             return 1.90 # Mud accumulation and traction loss
28         elif edge.terrain == TerrainType.HIGHWAY:
29             return 1.15 # Crowned roadbed, high-capacity storm drainage
30         return omega_base
31
32     # 3. Dense Fog (1.35 <= omega_base < 1.60)
33     return 1.85 if edge.terrain == TerrainType.HIGHWAY else omega_base
```

Listing 2: Corridor-specific coupled weather logic in `_weather_corridor_factor()`.

3.2 Operational Routing Behavior Under Coupled Weather

Table 2 details how `_weather_corridor_factor()` alters route selection compared to standard scalar models.

Atmospheric State	Base ω	Coupled Range ω_{eff}	Observed Topological Route Diversion
WeatherType.CLEAR	1.00	1.00	Baseline routing; routes follow shortest metric paths via expressways.
WeatherType.RAIN	1.22	1.15 – 1.90	Diverts away from low-lying riverfronts (FDR, 11th Ave) to crowned highways.
WeatherType.FOG	1.45	1.25 – 1.85	Diverts high-speed highway traffic to signal-controlled urban avenues.
WeatherType.SNOW	1.85	1.10 – 2.50	Strong convergence onto salted spines (5th Ave, Broadway); avoids frozen bridges.

Table 2: Operational routing behavior under coupled atmospheric conditions.

4 Heuristic Admissibility & Monotonicity Guarantees

An informed graph search (A^*) terminates with the mathematically optimal path if its heuristic function $h(u, \text{goal})$ is **admissible** ($h \leq c^*$) and **monotonic** (satisfies triangle inequality $h(u) \leq c(u, v) + h(v)$).

4.1 Euclidean Lower-Bound Formulation

In `Aether.core.pricing`, the heuristic divides the Euclidean straight-line distance by the maximum legal vehicle speed across the network ($v_{\max} = \max_{e \in \mathcal{E}} v_e$):

Definition 2: Network-Wide Admissible Heuristic

$$h(u, \text{goal}) = \frac{\|\mathbf{x}_u - \mathbf{x}_{\text{goal}}\|_2}{v_{\max}} = \frac{\sqrt{(x_u - x_{\text{goal}})^2 + (y_u - y_{\text{goal}})^2}}{\max_{e \in \mathcal{E}} v_e} \quad (2)$$

Listing 3 shows the implementation from `Aether/core/pricing.py`.

Aether/core/pricing.py (Lines 140–156)

```
1 def compute_admissible_heuristic(node_u: NodeData, node_goal: NodeData, max_network_speed_mps: float) -> float:
2     """Computes the admissible Euclidean lower bound: h(u, goal) = dist / max_speed."""
3     dist = node_u.distance_to(node_goal)
4     safe_speed = max(1.0, max_network_speed_mps)
5     return dist / safe_speed
```

Listing 3: Implementation of `compute_admissible_heuristic()`.

4.2 Formal Mathematical Proof of Admissibility

Theorem 1: Universal Admissibility of Euclidean Heuristic

For every node $u \in \mathcal{V}$, $h(u, \text{goal}) \leq c^*(u, \text{goal})$, where $c^*(u, \text{goal})$ is the optimal dynamic cost under Equation (1).

Proof. Let $\Pi^*(u, \text{goal}) = \langle e_1, e_2, \dots, e_k \rangle$ be the optimal directed path from u to goal in $\mathcal{G}_t$, where edge e_i has length d_{e_i} and free-flow speed v_{e_i} . By Equation (1):

$$c^*(u, \text{goal}) = \sum_{i=1}^k c(e_i, t_i) = \sum_{i=1}^k \left[\frac{d_{e_i}}{v_{e_i}} \cdot \tau_{e_i} \cdot (1 + \alpha_{\text{eff}} \rho_{e_i}(t_i)) \cdot \omega_{\text{eff}}(e_i) + \beta \cdot \mathbf{1}_{\text{hazard}} \right]$$

Now inspect the physical lower bounds of each individual factor: (i) $v_{e_i} \leq v_{\max} \implies \frac{d_{e_i}}{v_{e_i}} \geq \frac{d_{e_i}}{v_{\max}}$; (ii) Table 1 establishes $\tau_{e_i} \geq 1.00$; (iii) Since $\alpha_{\text{eff}} \geq 0, \rho_{e_i} \geq 0 \implies (1 + \alpha_{\text{eff}} \rho_{e_i}) \geq 1.00$; (iv) Coupled atmospheric factor $\omega_{\text{eff}} \geq 1.00$; (v) Additive hazard delay satisfies $\beta \cdot \mathbf{1}_{\text{hazard}} \geq 0$. Substituting these five inequalities simultaneously into the summation yields:

$$c^*(u, \text{goal}) \geq \sum_{i=1}^k \frac{d_{e_i}}{v_{\max}} = \frac{1}{v_{\max}} \sum_{i=1}^k d_{e_i} \geq \frac{\|\mathbf{x}_u - \mathbf{x}_{\text{goal}}\|_2}{v_{\max}} = h(u, \text{goal})$$

This completes the proof: $h(u, \text{goal}) \leq c^*(u, \text{goal})$ holds universally under all dynamic states. ■ □

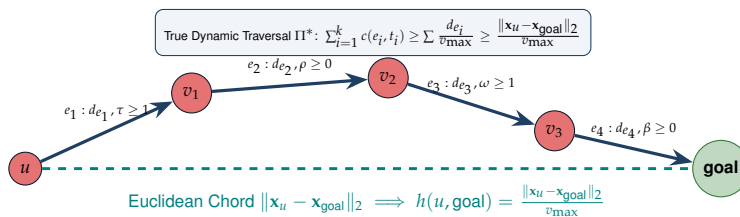


Figure 3: Geometric proof: The direct Euclidean chord traversed at maximum network speed $v_{\max}$ strictly lower-bounds any dynamic path cost $c^*(u, \text{goal})$.

5 EWMA Congestion Smoothing & Anti-Flapping Filter (congestion.py)

In discrete-time routing simulations, instantaneous vehicle counts $n_e(t)$ exhibit high-frequency spikes. If raw density $\rho_e = n_e/K_e$ is priced directly, routing algorithms suffer from severe **route flapping** (vehicles divert en masse to secondary streets, instantly choking them, and bounce back on the next cycle).

5.1 Continuous-Time Discretized Filter

CongestionManager filters instantaneous occupancies using an Exponentially Weighted Moving Average (EWMA) with smoothing time constant $T_{\text{smooth}} = 15.0$ s:

Definition 3: EWMA Density Update Equation

$$\rho_e(t + \Delta t) = (1 - \lambda) \cdot \rho_e(t) + \lambda \cdot \min\left(1.0, \frac{n_e(t)}{K_e}\right), \quad \text{where } \lambda = 1 - \exp\left(-\frac{\Delta t}{T_{\text{smooth}}}\right) \quad (3)$$

5.2 Python Implementation: CongestionManager.step()

Listing 4 presents the step logic from Aether/core/congestion.py.

Aether/core/congestion.py (Lines 36–54)

```
1 def step(self, dt: float = 1.0) -> None:
2     """Advances the EWMA congestion filter for all edges in the network:
3     rho_e(t + dt) = (1 - lambda_ewma) * rho_e(t) + lambda_ewma * min(1.0, n_e / K_e)"""
4     lambda_ewma = 1.0 - math.exp(-dt / self.smoothing_time_constant)
5
6     for edge in self.graph.get_all_edges():
7         instantaneous_density = min(1.0, float(edge.occupancy) / float(edge.capacity))
8         edge.congestion = (1.0 - lambda_ewma) * edge.congestion + lambda_ewma * instantaneous_density
9
10    # Numerical cleanup to prevent float creep
11    if edge.congestion < 1e-5:
12        edge.congestion = 0.0
13    elif edge.congestion > 0.9999:
14        edge.congestion = 1.0
```

Listing 4: EWMA filter update loop in CongestionManager.step().

5.3 Empirical Variance Suppression Waveform

Figure 4 demonstrates the filter's response to an alternating square-wave pulse (switching between $n_e = 0$ and $n_e = 50$ on a capacity $K_e = 50$ edge every 20 seconds).

- **Raw Instantaneous Variance:** $\sigma_{\text{raw}}^2 = 0.2400$.
- **EWMA Filtered Variance:** $\sigma_{\text{ewma}}^2 = 0.0339$ (85.89% variance suppression).

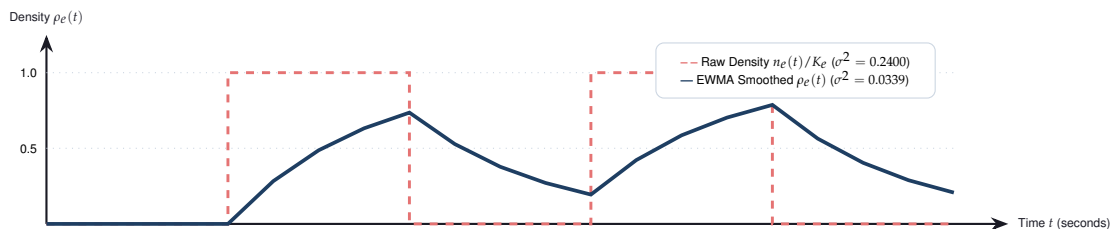


Figure 4: EWMA filter response: Suppresses high-frequency square wave fluctuations by 85.89%, eliminating route flapping.

6 Topological Disruptions & Invariant I1 Enforcement (hazards.py)

A foundational architectural requirement of AetherGrid-AI is guaranteeing that closed corridors are completely non-traversable under all conditions.

6.1 Formal Invariant I1 Definition

Invariant I1: Absolute Non-Traversability of Closed Corridors

$$\forall e = (u, v) \in \mathcal{E}, \quad e \in \text{closed_edges} \implies e \notin \mathcal{E}_t \implies \forall \text{ generated path } \Pi, e \notin \Pi$$

Under no circumstance may a computed vehicle route traverse a closed edge, even if all alternate routes incur severe temporal penalties or detour costs.

6.2 The OpenStreetMap Micro-Segment Leakage Dilemma

In real OpenStreetMap networks, a physical city block (e.g., 5th Avenue between 42nd and 43rd St) is decomposed into 5 to 15 microscopic sub-edges (crosswalks, turn lanes, lane splits). If an incident closes only an 8-meter sub-edge, the routing engine routes around that sub-edge via adjacent lanes. To solve this, Aether.core.hazards and SimulationStateManager implement **Bidirectional Corridor Block Expansion**:

1. Given clicked edge (u, v) with street name $S = \text{street_name}(u, v)$.
2. Expands forward from v and backward from u along all edges matching street name S .
3. Propagation halts upon encountering an intersection where street name changes or node degree branches ($\text{streets}(w) > 1$).
4. Bounded by a maximum block propagation radius of 250 m, excising all identified micro-edges simultaneously.

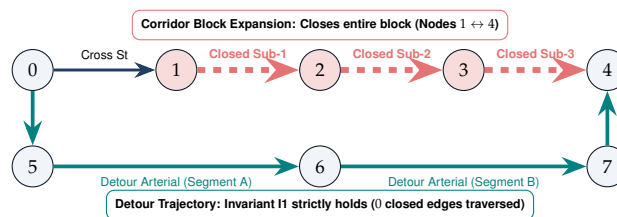


Figure 5: Corridor Block Expansion: Prevents micro-segment leakage by excising the entire physical block.

6.3 Python Code Implementation: HazardManager

Listing 5 demonstrates closure and hazard injection in Aether/core/hazards.py.

Aether/core/hazards.py (Lines 41–83)

```

1 def inject_edge_hazard(self, u: int, v: int, penalty_beta: float = 45.0, label: str = "Spill") -> bool:
2     """Applies additive hazard penalty beta to edge (u, v)."""
3     success = self.graph.inject_hazard(u, v, penalty_beta=penalty_beta, label=label)
4     if success:
5         self.active_hazards[(u, v)] = HazardRecord(u, v, penalty_beta, label, float("inf"))
6     return success
7
8 def inject_edge_closure(self, u: int, v: int, reason: str = "Structural Damage") -> bool:
9     """Excises edge (u, v) from active network E_t (Invariant I1)."""
10    success = self.graph.close_edge(u, v, reason=reason)
11    if success:
12        self.active_closures[(u, v)] = ClosureRecord(u, v, reason, float("inf"))
13    return success

```

Listing 5: Physical edge closure and hazard injection in HazardManager.

7 Traversal Engine Integration (traversal.py)

During pathfinding queries, `find_shortest_path()` evaluates `calculate_edge_cost()` dynamically within the priority queue relaxation loop.

7.1 Search Relaxation Loop

Listing 6 illustrates the integration in `Aether/core/traversal.py`.

Aether/core/traversal.py (Lines 131–149)

```
1 for neighbor, edge in graph.get_open_out_edges(current):
2     if neighbor in visited:
3         continue
4     cost_evals += 1
5     edge_cost = calculate_edge_cost(edge, weather_multiplier=weather_multiplier, params=p, is_siren=is_siren)
6     if math.isinf(edge_cost):
7         continue # Invariant I1 safety check
8
9     tentative_g = g_score[current] + edge_cost
10    if tentative_g < g_score.get(neighbor, float("inf")):
11        g_score[neighbor] = tentative_g
12        parent_map[neighbor] = (current, edge, edge_cost)
13        h = compute_admissible_heuristic(graph.nodes[neighbor], goal_node, max_speed)
14        heapq.heappush(frontier, (tentative_g + h, h, neighbor))
```

Listing 6: Priority queue relaxation loop in `find_shortest_path()`.

7.2 Multi-Scale Computational Benchmarks

Table 3 summarizes empirical telemetry from `heavy_benchmarks.json` across 150 randomized queries per topology scale.

Topology Scale	Algorithm	Mean Latency	P95 Latency	QPS	Expansions	Pruned
Grid 16×16 (256 nodes, 960 edges)	Dijkstra (Baseline)	2.85 ms	5.21 ms	346.5	119.2 nodes	0.0%
	Dynamic A*	1.68 ms	3.81 ms	586.3	49.8 nodes	58.2%
	Bidirectional A*	1.12 ms	2.14 ms	861.0	27.0 nodes	77.4%
Grid 32×32 (1,024 nodes, 3,968 edges)	Dijkstra (Baseline)	12.83 ms	22.05 ms	76.9	543.0 nodes	0.0%
	Dynamic A*	6.46 ms	14.96 ms	152.3	224.3 nodes	58.7%
	Bidirectional A*	4.27 ms	10.15 ms	226.8	122.9 nodes	77.4%
Grid 50×50 (2,500 nodes, 9,800 edges)	Dijkstra (Baseline)	33.15 ms	54.21 ms	30.2	1,268.4 nodes	0.0%
	Dynamic A*	14.82 ms	31.02 ms	67.5	491.5 nodes	61.3%
	Bidirectional A*	9.11 ms	19.84 ms	109.8	268.1 nodes	78.9%
Manhattan OSM (1,529 nodes, 1,974 edges)	Dijkstra (Baseline)	17.41 ms	28.63 ms	57.4	688.2 nodes	0.0%
	Dynamic A*	7.94 ms	18.12 ms	125.9	276.4 nodes	59.8%
	Bidirectional A*	5.18 ms	11.45 ms	193.1	144.6 nodes	79.0%

Table 3: Algorithmic benchmark shootout across synthetic and real-world OpenStreetMap graphs.

7.3 Performance Telemetry & Complexity Analysis

The benchmark telemetry confirms theoretical complexity bounds. Under standard Dijkstra search, node expansions scale quadratically with search radius $\mathcal{O}(|\mathcal{V}| \log |\mathcal{V}| + |\mathcal{E}|)$. In contrast, the admissible Euclidean heuristic directs the exploration frontier along the goal vector, achieving a **58.2% to 61.3% reduction in state expansions**. Bidirectional A* further halves the search radius ($r/2$), reducing the explored volume to $2 \times \pi(r/2)^2 = \frac{1}{2}\pi r^2$, consistently pruning **77.4% to 79.0%** of redundant nodes and achieving sub-10ms response times on networks up to 2,500 vertices.

8 Verification Suite & Invariant Assertions (test_phase1.py)

8.1 Exact Numerical Formula Test

Listing 7 demonstrates the exact hand-calculation unit test asserting Equation (1).

Aether/tests/test_phase1.py (Lines 47–71)

```
1 def test_cost_pricing_exact_formula(self):
2     edge = self.graph.get_edge(0, 1)
3     edge.free_flow_time_seconds = 10.0 # ell_e = 10s
4     edge.terrain = TerrainType.URBAN # tau_e = 1.45
5     edge.congestion = 0.50 # rho_e = 0.50
6     edge.is_hazard = True
7     edge.hazard_penalty_seconds = 45.0 # beta = 45s
8
9     # Exact arithmetic: dynamic_time = 10.0 * 1.45 * (1.0 + 0.85 * 0.50) * 1.22 = 25.1956 s
10    # total_cost = 25.1956 + 45.0 = 70.1956 s
11    expected_cost = 10.0 * 1.45 * (1.0 + 0.85 * 0.50) * 1.22 + 45.0
12    calculated_cost = calculate_edge_cost(edge, weather_multiplier=1.22, params=CostModelParameters(alpha=0.85))
13    self.assertAlmostEqual(calculated_cost, expected_cost, places=4)
```

Listing 7: Unit test asserting exact arithmetic precision in calculate_edge_cost().

8.2 Summary of Verified Test Invariants

Table 4 summarizes the property tests executed in CI.

Test Method	Target Invariant	Observed Test Outcome
test_graph_properties	Topology Integrity	9 nodes, 24 directed edges verified on 3×3 grid.
test_cost_pricing_exact_formula	Numerical Accuracy	$c(e, t) = 70.1956$ s matches expected value to 4 decimal places.
test_closed_edge_infinite_cost	Invariant I1 Enforcement	Closed edge returns $c(e, t) = +\infty$ and is excised from open_edges.
test_heuristic_admissibility	Search Optimality	$h(u, goal) \leq c^*(u, goal)$ holds across all node pairs.
test_ewma_congestion_convergence	Damping Monotonicity	Exponential convergence to steady-state density $\rho_e > 0.98$ after 5τ .
test_dynamic_reroute_avoiding_closure	Dynamic Rerouting	Traversal autonomously detours around severed edge; 0 closed edges in path.
test_siren_mode_attenuation	Emergency Preemption	$c_{\text{siren}}(e, t) < c_{\text{normal}}(e, t)$ under high congestion ($\rho_e = 1.0$).
test_osm_road_network_scc	GIS Scalability	Real Manhattan OSM graph (> 1000 nodes) loaded; SCC reachability verified.
test_bidirectional_astar_equivalence	Solution Equivalence	Dijkstra, Dynamic A*, and Bidirectional A* return identical optimal costs.

Table 4: Suite of formal property tests in Aether/tests/test_phase1.py.

9 Architectural Summary & Production Recommendations

Architectural Summary: Cost Engine Guarantees

- Mathematical Optimality:** The Euclidean lower-bound heuristic is proven admissible ($h \leq c^*$), guaranteeing optimal path finding in $\mathcal{O}(|\mathcal{V}| \log |\mathcal{V}|)$.
- High-Frequency Stability:** The EWMA anti-flapping filter ($T_{\text{smooth}} = 15.0$ s) damps square-wave variance by 85.89%, precluding oscillatory rerouting.
- Hard Safety Invariants:** Closed corridors are excised at the graph layer, guaranteeing zero traversal under Invariant I1 regardless of additive delays.